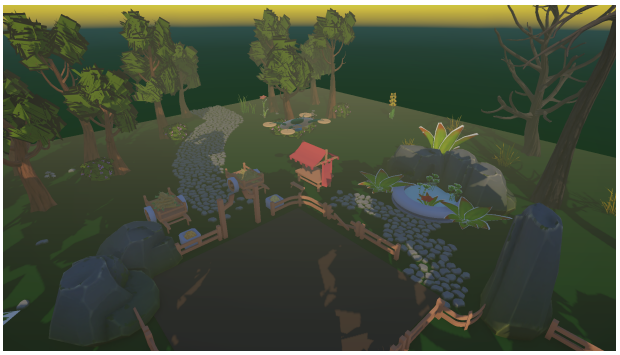
Le Potager Solarpunk

Fiche de préparation à l'oral · Projet 2 Unity · Kerouann Touami-Causse · BUT2 Informatique

1. Le jeu en 30 secondes

Le joueur pilote un drone horticole à propulseurs magnétiques au-dessus d'un potager solarpunk. Il prend des graines aux silos, les plante sur les parcelles de terre, les arrose (réservoir rechargé à la fontaine) et doit récolter au bon moment : trop tôt elles ne sont pas prêtes, trop tard elles éclatent. Les légumes aspirés vont dans la soute, puis dans la charrette du bon légume pour un bonus. La partie dure **5 minutes** : on gagne en atteignant le score cible **et** le nombre de récoltes demandé pour chaque légume (réglés dans la GameSession, actuellement 250 points et 50 de chaque). Le meilleur score est conservé d'une partie à l'autre.





À gauche : le potager décoré. À droite : le jeu en cours, avec l'interface (score, chrono, objectifs, soute, eau, graine sélectionnée).

2. Commandes

Action	Clavier / souris	Manette
Se déplacer	ZQSD / flèches	Stick gauche
Tourner / incliner la vue	Souris	Stick droit
Monter / descendre	Espace / Ctrl gauche	RB / LB
Planter	Clic gauche	Gâchette droite (RT)
Changer de graine	E / A ou molette	Croix directionnelle → / ←
Arroser	Clic droit (maintenu)	Gâchette gauche (LT)
Déposer dans la charrette	R	A (bouton bas)
Recharger eau / graines	R maintenu (fontaine / silo)	A maintenu
Pause	Échap	Start

Toutes les commandes passent par un seul asset **Input Actions** : chaque action (Move, Plant…) regroupe les touches clavier ET les boutons manette. Le code lit l'action, jamais une touche précise.

3. Architecture : un script = une responsabilité

Script	Posé sur	Responsabilité
DroneController	Drone	Déplacer le drone (relatif à la caméra), gérer l'altitude, rester dans le potager
CameraOrbit	Main Camera	Suivre le drone et tourner autour (souris / stick droit)
SeedPlanter	Drone	Planter une graine sous le drone si la place le permet
SoilPlot	Chaque parcelle	Marquer la terre cultivable, limiter le nombre de plantes

Script	Posé sur	Responsabilité
Plant	Prefab de la plante (Ble)	Croissance, arrosage, récolte par aspiration, éclatement
WaterSprayer	Drone	Jet de particules + arroser les plantes sous le drone
ScoreManager	GameManager	Compter les points et les récoltes par plante (Dictionary), prévenir par événements
DroneCargo	Drone	Soute : légumes transportés (capacité limitée), par type
DroneSeeds	Drone	Stock de graines par légume, consommé à la plantation
DroneInteractor	Drone	Détecte bacs, fontaine et silos (OnTriggerEnter/Exit) ; R pour déposer ou recharger
DepositBin	Charrettes	Accepte un type de légume, donne un bonus, anime le dépôt
WaterRefillStation / SeedBin	Fontaine / silos	Fournissent de l'eau / des graines à un débit réglable
DroneStatusUI	HUD (Canvas)	Jauge d'eau (Image Filled), contenu de la soute, aide contextuelle
PlantSelector	Drone	Liste des plantes disponibles, graine sélectionnée, changement avec les touches
PlantSelectorUI	HUD (Canvas)	Afficher la graine sélectionnée (nom, points, position)
HarvestUI	HUD (Canvas)	Afficher le score total et le nombre de récoltes par plante
DroneHover	DroneVisuel	Vol stationnaire (flottement sinusoïdal) et inclinaison selon la vitesse
SoundPlayer	GameManager	Jouer les sons ponctuels (PlayOneShot) avec une légère variation de hauteur
GameSession	GameManager	Chrono, objectifs, victoire / fin du temps, passage à la scène de fin
ObjectivesUI	HUD (Canvas)	Afficher le chrono et l'avancement des objectifs
PauseMenu	HUD (Canvas)	Pause (timeScale), reprendre, recommencer, retour au menu
TitleMenu / EndMenu	Scènes MenuPrincipal / Fin	Boutons Jouer, Quitter, Rejouer, Menu ; affichage du score et du record
BestScore / GameResults	Classes statiques	Meilleur score (PlayerPrefs) / résultat transmis à la scène de fin
SoundSettings / SoundSettingsUI	Statique / menus	Volumes Général, Musique, Effets et Muet, sauvegardés, appliqués à l'AudioMixer
Outils d'éditeur (dossier Editor)	—	Construire les menus, les tas de récolte, nettoyer, faire la build, prendre les captures

Organisation : tout le contenu du jeu est dans **Assets/_Project**, rangé par type (Scripts, Prefabs, Materials, Scenes, Input, Audio, VFX, UI, Models, Animations). Tous les réglages (vitesses, durées, limites, prefabs) sont des champs **[SerializeField]** modifiables dans l'Inspector, avec une infobulle (Tooltip).

4. Concepts techniques : questions probables et réponses

Déplacement et commandes (jalon 1)

Q : Comment le même code marche-t-il au clavier et à la manette ?
Le script reçoit une `InputActionReference` (l'action *Move*) et lit `ReadValue<Vector2>()`. C'est l'Input System qui fusionne ZQSD, flèches et stick dans une seule valeur entre -1 et 1. Aucun script différent par périphérique.

Q : Pourquoi activer les actions dans OnEnable et les désactiver dans OnDisable ?
C'est le cycle de vie d'un composant Unity : l'action n'écoute les touches que lorsque le composant est actif. On se désabonne aussi des événements dans OnDisable, sinon une action pourrait appeler un objet détruit.

Q : Pourquoi multiplier par Time.deltaTime ?
Update est appelé une fois par image. Sans deltaTime, un PC à 144 FPS déplacerait le drone 2,4 fois plus vite qu'un PC à 60 FPS. Avec deltaTime, la vitesse s'exprime en unités par seconde.

Q : Pourquoi la souris n'utilise-t-elle PAS Time.deltaTime ?
La souris renvoie un déplacement en pixels depuis la dernière image : c'est déjà une quantité par image. Le stick renvoie une position (-1 à 1) qu'on transforme en vitesse : lui doit être multiplié par deltaTime.

Q : Pourquoi la caméra est-elle mise à jour dans LateUpdate ?
LateUpdate s'exécute après tous les Update. Le drone a donc déjà bougé quand la caméra se place : elle suit sa position finale, sans tremblement.

Q : Comment le drone reste-t-il dans le potager et au-dessus du sol ?
Après chaque déplacement, `Mathf.Clamp` borne X et Z entre les limites du terrain, et l'altitude entre une hauteur minimale (2 m) et maximale (8 m). Le drone n'a pas de collision physique avec le sol : il vole, on impose directement sa position.

Q : Pourquoi le drone est-il un objet vide avec un visuel en enfant ?
Le code déplace le parent « Drone » (qui porte aussi le collider et le Rigidbody) ; le visuel « DroneVisuel » est un enfant que l'animation de vol stationnaire peut faire bouger. Animation et gameplay ne s'écrasent pas mutuellement, et la zone de détection ne tremble pas (piste de la brique 4.1).

Plantation et croissance (jalon 2)

Q : Comment la graine apparaît-elle exactement sous le drone ?
Un `Physics.Raycast` est lancé vers le bas depuis le drone. La graine est instanciée au point d'impact (`hit.point`) : elle tombe sous le drone, pas devant ni à côté.

Q : Comment sait-on qu'on est au-dessus de la terre ?
Chaque parcelle porte un composant `SoilPlot`. Si le collider touché par le rayon n'en a pas (herbe), on ne plante pas. Un composant est plus sûr qu'un Tag : aucune chaîne de caractères à mal orthographier.

Q : Pourquoi un Prefab ?
La plante est créée des dizaines de fois avec `Instantiate`. Modifier le Prefab modifie toutes les futures plantes.

Q : Comment gérer plusieurs plantes en même temps ?
Une variable simple ne garde qu'un objet ; on stocke donc les plantes dans une `List<Plant>`. Avant de l'utiliser, `RemoveAll(p => p == null)` retire celles qui ont été détruites (récoltées ou éclatées). La liste sert à vérifier l'espacement minimum et le nombre maximum par parcelle.

Q : Comment la plante attend-elle sans bloquer le jeu ?
La croissance est une **coroutine** (`IEnumerator Start()`). À chaque image, elle ajoute `Time.deltaTime × growthSpeed` au temps écoulé, met à jour la taille, puis `yield return null` rend la main au moteur jusqu'à l'image suivante.

Q : Pourquoi un enum GrowthStage plutôt que des booléens ?
Une plante est dans un seul état à la fois : Seed, Ripe ou Overripe. Une seule variable d'état évite les combinaisons impossibles (isSeed et isRipe vrais en même temps). Seul l'état Ripe est récoltable.

Q : Pourquoi deux graines plantées à quelques secondes d'écart ne mûrissent-elles pas ensemble ?
Chaque plante a sa propre coroutine qui démarre à sa plantation, et une petite variation aléatoire (±15 %) est appliquée à ses durées.

Q : Comment la plante disparaît-elle ?
`Destroy(gameObject)` la retire de la scène et de la Hierarchy à la fin de l'étape « trop mûr ».

Arrosage, récolte et score (jalons 2 et 3)

Q : Comment une plante sait-elle qu'elle est arrosée ?
Tant que le bouton est maintenu (`IsPressed()`), le drone fait un `Physics.OverlapCapsule` : une colonne invisible sous lui. Il appelle `Water()` sur chaque plante trouvée. La plante note jusqu'à quand elle reste humide ; tant qu'elle l'est, son temps de croissance avance × `wateredGrowthMultiplier` (2,5 par défaut, réglable).

Q : Pourquoi IsPressed() pour arroser et l'événement performed pour planter ?
Planter est une action ponctuelle : un appui = une graine (performed). Arroser est continu : on vérifie à chaque image si le bouton est encore maintenu (IsPressed).

Q : Le drone vole haut et le blé est au sol : comment se touchent-ils ?
Le blé porte une Box Collider en Is Trigger très haute (9 m) : une colonne invisible qui monte jusqu'au plafond de vol du drone. Le drone a un Rigidbody en Is Kinematic (déplacé par le code, pas par la physique) : c'est nécessaire pour que Unity déclenche les événements de trigger.

Q : Pourquoi OnTriggerStay et pas OnTriggerEnter ?
Si le drone est déjà au-dessus quand le blé devient mûr, Enter a eu lieu trop tôt. Stay est appelé tant que le drone reste dans la zone. Et comme on ne stocke aucune référence au drone, il n'y a rien à vider dans OnTriggerExit : aucune erreur si le blé éclate pendant le survol.

Q : Comment le blé, créé pendant la partie, connaît-il le ScoreManager ?
Par **injection de dépendance** : juste après `Instantiate`, le SeedPlanter appelle `newPlant.Initialize(scoreManager)`. On ne peut pas glisser cette référence dans l'Inspector du Prefab, car le ScoreManager est dans la scène.

Q : Comment l'affichage du score est-il mis à jour ?
Le ScoreManager déclenche un événement `ScoreChanged` ; le ScoreUI s'y abonne dans OnEnable et se désabonne dans OnDisable. Le ScoreManager ne connaît pas l'interface : chaque script garde une seule responsabilité.

Q : Comment l'interface reste-t-elle à sa place en 16:9, large et 4:3 ?
Le Canvas a un Canvas Scaler en Scale With Screen Size (référence 1920×1080, Match 0,5). Le texte est ancré en haut à gauche avec un pivot (0, 1) : il reste collé au coin quelle que soit la taille de l'écran. Un contour (Outline) le garde lisible sur le décor.

Q : Comment fonctionne l'aspiration à la récolte ?
Les points sont donnés immédiatement (pas de délai visible), puis `StopAllCoroutines()` arrête la croissance et une coroutine fait monter le blé vers la position actuelle du drone en tournant et en rétrécissant (interpolation `Vector3.Lerp` avec une accélération t^2), avant `Destroy`.

Plusieurs plantes et affichage (bonus « Légumes inégaux »)

Q : Comment chaque plante a-t-elle ses propres règles ?
Tout est défini sur le Prefab de la plante (composant Plant) : nom affiché, durées de pousse, points, nombre maximum par parcelle, espacement. Ajouter une plante = créer un Prefab, sans toucher au code. La parcelle demande simplement à la plante s'il reste de la place (HasRoomFor).

Q : Comment la pousse est-elle rendue réaliste ?
La plante affiche une liste de modèles de pousse (ex. Wheat_1 puis Wheat_2) un par un selon l'avancement, puis le modèle mûr, puis le modèle tombé (trop mûr). Un seul modèle est visible à la fois (SetActive).

Q : Comment change-t-on de graine ?
Le PlantSelector garde un tableau de Prefabs (`Plant[]`) et un index. Les actions NextPlant / PreviousPlant changent l'index en bouclant avec un modulo ; un événement `SelectionChanged` prévient l'interface. Le SeedPlanter plante `plantSelector.CurrentPlant`.

Q : Comment compte-t-on les récoltes par plante ?
Le ScoreManager utilise un `Dictionary<string, int>` : clé = nom de la plante, valeur = quantité. HarvestUI s'abonne aux événements et reconstruit le texte avec un StringBuilder.

Gestion des ressources (mécaniques personnelles)

Q : Quelles ressources le joueur doit-il gérer ?
L'eau (réservoir de 100, l'arrosage en consomme 20/s, recharge à la fontaine), les graines (un stock par légume, 1 graine par plantation, recharge au silo du légume) et la soute (6 légumes max : pleine = plus de récolte tant qu'on n'a pas déposé dans la charrette du bon légume, qui donne un bonus de points).

Q : Comment le drone sait-il qu'il est au-dessus d'une charrette ou de la fontaine ?
Chaque élément a une Box Collider en trigger haute. Le DroneInteractor mémorise l'élément survolé dans OnTriggerEnter et l'oublie dans OnTriggerExit (seulement si c'est bien celui qu'on quitte) : le « duo inséparable » de la fiche. En secours, au moment d'appuyer sur R, il cherche aussi avec un OverlapCapsule sous le drone.

Q : Pourquoi le score reste-t-il donné à la récolte et pas au dépôt ?
La brique 3.2 exige que le score monte à chaque récolte, sans délai visible. Le dépôt donne seulement un bonus en plus.

Q : Pourquoi [RequireComponent(typeof(Collider))] sur les bacs ?
Un bac sans collider ne pourrait jamais être détecté : Unity refuse d'ajouter le script tant qu'un collider n'est pas présent, ce qui évite l'erreur de configuration.

Q : Comment le stock de graines se remplit-il « petit à petit » ?
Le stock est un float : on ajoute débit × Time.deltaTime à chaque image tant que R est maintenu, sans dépasser la capacité de la plante. L'interface n'est prévenue que quand le nombre entier change.

Animations (jalon 4)

Q : Comment la croissance est-elle animée ?
Une seule animation « PopPlante » (échelle 0 → 115 % → 100 %, Loop Time décoché) dans un Animator Controller « AC_PopPlante », posé sur chaque conteneur de stade. Quand le script active un stade (SetActive), l'Animator rejoue automatiquement son état par défaut : le stade apparaît avec un rebond. Aucune ligne de code.

Q : Pourquoi l'Animator n'est-il pas sur la racine de la plante ?
La racine est déplacée et rétrécie par le code pendant l'aspiration de la récolte. Si l'Animator animait aussi son échelle, les deux s'écraseraient. L'animation agit donc uniquement sur les enfants (conteneurs de stade).

Sons, particules et ambiance (jalon 4)

Q : Comment le drone flotte-t-il ?
Le script DroneHover, posé sur le visuel (enfant), ajoute `sin(temps)` à sa hauteur locale (8 cm sur 3 s) et l'incline dans le sens de la vitesse avec `Quaternion.Slerp`. Une première version avec l'Animator saccadait : le code donne un mouvement continu et réglable.

Q : Comment les sons sont-ils organisés ?
Toutes les AudioSource passent par un **AudioMixer** avec deux groupes : Musique et SFX. Les sons ponctuels (planter, récolter, déposer) sont joués avec PlayOneShot et une petite variation de hauteur (pitch) pour ne pas être répétitifs ; les sons continus (arrosage, recharge) sont des boucles qu'on lance et coupe.

Q : Comment les particules sont-elles faites ?
Avec le Particle System de Unity : jet d'eau sous le drone, effets magnétiques des propulseurs, éclats à la récolte et à l'éclatement (Bursts, Color et Size over Lifetime). Les effets de récolte sont des Prefabs instanciés puis détruits après leur durée.

Écrans, pause et meilleur score (jalon 5)

Q : Comment passe-t-on d'un écran à l'autre ?
Trois scènes : MenuPrincipal, Game et Fin, déclarées dans cet ordre dans la build. Les boutons appellent `SceneManager.LoadScene(nom)`. Les noms de scènes sont des champs de l'Inspector, pas écrits en dur dans plusieurs scripts.

Q : Comment la pause fige-t-elle le jeu ?
`Time.timeScale = 0` : tout ce qui utilise Time.deltaTime s'arrête (croissance, chrono, déplacements). On désactive aussi la map d'actions « Player » (sinon on pourrait planter en pause), on libère le curseur, et on remet timeScale à 1 avant de changer de scène.

Q : Comment le meilleur score est-il conservé ?
Avec `PlayerPrefs` (sauvegarde clé / valeur de Unity). BestScore.TrySubmit ne l'écrit que si le nouveau score est plus grand : le record ne baisse jamais.

Q : Comment l'écran de fin connaît-il le score, alors que la scène de jeu est détruite ?
La classe statique GameResults garde le dernier score, la victoire ou non, le temps et « nouveau record » : une variable statique survit au chargement d'une nouvelle scène.

Q : Comment fonctionnent les réglages du son ?
Les volumes des groupes du mixer sont des **paramètres exposés** (VolumeGeneral, VolumeMusique, VolumeSFX). Le curseur va de 0 à 1 et on le convertit en décibels avec 20 × log10(valeur), car l'oreille perçoit le volume de façon logarithmique ; Muet = −80 dB. Les réglages sont sauvegardés dans PlayerPrefs.

Q : Comment les menus marchent-ils à la manette ?
L'EventSystem utilise l'InputSystemUIInputModule et a un « First Selected » : un bouton est sélectionné dès l'ouverture, et le stick / la croix naviguent entre les boutons.

Finition, build et outils (jalon 6)

Q : Qu'avez-vous nettoyé ?
Suppression des actions d'entrée par défaut inutilisées (Look, Jump, Attack…), noms clairs dans la Hierarchy (Parcelle1…4, Silo, Charrette, Fontaine), dossier Docs supprimé, Console sans erreur ni avertissement.

Q : Pourquoi des scripts dans un dossier « Editor » ?
Les scripts placés dans un dossier Editor ne sont pas inclus dans le jeu : ce sont des outils pour Unity (menu « Solarpunk ») qui construisent les écrans, les tas de récolte, nettoient le projet, prennent les captures et font la build Windows avec son zip pour itch.io.

Q : Comment les tas de récolte des charrettes sont-ils faits ?
Un outil empile des copies des modèles Quaternius (carotte, gerbe de blé) en dôme, avec des positions et rotations aléatoires mais reproductibles (graine fixe), puis enregistre le tout en Prefab et le pose à la bonne taille sur le plateau de la charrette (repéré par un rayon lancé du dessus).

5. Choix personnels à savoir justifier

- **Caméra orbitale** (souris / stick droit) au lieu d'une caméra fixe : elle démarre en vue de dessus (90°) comme demandé, mais le joueur peut tourner et incliner la vue. Le déplacement reste intuitif car il est relatif à la caméra : « haut » = haut de l'écran.
- **Altitude contrôlable** (Espace / Ctrl, RB / LB). ⚠ La brique 1.1 demande une hauteur constante : justification = le drone ne s'enfonce jamais (plancher à 2 m, plafond à 8 m) et la hauteur reste bornée et maîtrisée. Point de vigilance pour la récolte : la zone de détection des légumes doit monter jusqu'au plafond.
- **Parcelles** : on ne plante que sur la terre, avec un nombre maximum de plantes par parcelle et un espacement minimum, pour un potager lisible et une vraie gestion de l'espace.
- **Thème** : le blé, avec trois visuels distincts (graine, blé mûr, blé trop mûr). Durées : 10 s pour mûrir, 7 s de fenêtre de récolte, 4 s avant d'éclater.
- **Récolte par aspiration** : au lieu de disparaître d'un coup, le blé s'envole en tourbillonnant vers le drone (proche du défi bonus « Juiciness »).
- **Commandes manette** : stick gauche = déplacement, stick droit = caméra, gâchettes = planter (RT) / arroser (LT), boutons de tranche = altitude.
- **Plusieurs plantes** (blé, carotte…) sélectionnables avec affichage à l'écran, chacune avec ses durées, ses points et sa capacité par parcelle → défi bonus « Légumes inégaux ».
- **Pousse réaliste** : vrais modèles 3D par stade au lieu d'un objet qui grossit.
- **Drone sur mesure** : corps ovale, bras arrondis, propulseurs magnétiques (plaques cuivrées avec halo lumineux, pas d'hélices) et trois boîtes sous le ventre (graines, eau, stockage). Caméra orbitale rapprochée (distance réglable dans l'Inspector).
- **Gestion de ressources** : réservoir d'eau limité (fontaine), graines limitées par légume (silos), soute limitée et charrettes de dépôt par légume avec animation (les légumes tombent du drone, la charrette rebondit).
- **Fin de partie à double objectif** : score cible ET quantité de chaque légume, en 5 minutes. Les valeurs sont réglables dans l'Inspector de la GameSession pour équilibrer la difficulté.
- **Réglages du son** (Général, Musique, Effets, Muet) dans le menu titre et le menu pause, sauvegardés d'une partie à l'autre.
- **Décor** : jardin entouré d'arbres, chemins de pierres, clôtures, échoppe, fontaine, rochers et fleurs ; tas de carottes et de blé sur les charrettes.

6. Avancement

Jalon	Contenu	État
1	Le drone survole le potager	Terminé
2	Planter (2.1) · Pousser (2.2) · Arroser (2.3)	Terminé
3	Récolte (trigger, aspiration) · Score à l'écran	Terminé
4	Animations · Sons et particules · Décor solarpunk	Terminé
Bonus	Plusieurs plantes (Légumes inégaux) · récolte par aspiration · eau, graines, soute et dépôts	Fait
5	Écran titre / fin / pause · Meilleur score (PlayerPrefs) · réglages du son	Terminé

Jalon	Contenu	État
6	Rangement · Debug · Build et itch.io · Dépôt Git	Terminé : build Windows publiée sur itch.io (rb6'solarpunk), dépôt GitHub public

Livraison (deux liens) · Page itch.io (build jouable) : **https://rb6-k.itch.io/rb6solarpunk** · Dépôt de code (public, sans Library/Temp/Logs/Build) : **https://github.com/kerker48/Solarpunk**

7. Crédits des assets

Asset	Auteur	Licence	Utilisé pour
Ultimate Crops Pack	Quaternius (quaternius.com · patreon.com/quaternius)	CC0 1.0 Universal (domaine public)	Le blé à ses différents stades (Wheat_1 à Wheat_4, Wheat_Crop)
Fantasy Town Kit (2.0)	Kenney (kenney.nl)	CC0 1.0 Universal (domaine public)	Fontaine, charrettes, silos (cheminées), décor
Stylized Nature MegaKit (version Standard gratuite)	Quaternius (quaternius.com)	CC0 1.0 Universal (domaine public)	Décor végétal : arbres, buissons, herbes, fleurs, cailloux
Drone solarpunk et tas de graines	Modèles créés pour le projet (générés par script)	—	Drone (corps, bras, propulseurs, boîtes), tas de graines des silos
Effets de particules	Créés dans Unity (Particle System)	—	Jet d'eau, effet magnétique, récolte, éclatement
« Champ de tournesol »	Komiku	CC0	Musique de fond
Interface Sounds · RPG Audio	Kenney (kenney.nl)	CC0 1.0 Universal (domaine public)	Sons d'interface, de récolte et de dépôt
100 CC0 SFX #2	rubberduck (OpenGameArt)	CC0	Sons d'eau et d'ambiance

Source : fichiers License.txt et License_Standard.txt fournis avec les packs. La licence CC0 n'oblige pas à créditer, mais la fiche du projet l'exige : ces crédits doivent figurer sur la page itch.io (brique 6.3). Formulation possible : « Modèles 3D : Ultimate Crops Pack et Stylized Nature MegaKit par Quaternius, Fantasy Town Kit par Kenney (CC0) ».

8. Déroulé conseillé de la démonstration

- Montrer le projet : dossier **_Project** rangé, Hierarchy lisible, Console vide.
- Dans l'Inspector du Drone : vitesse, altitude min/max ; sur le Prefab Plant : les durées de croissance (moins de 30 s pour les trouver).
- Play Mode : déplacement clavier puis manette, rotation de la caméra, montée / descente bloquée à 2 m.
- Planter sur une parcelle, essayer sur l'herbe (refus), essayer trop près (refus), remplir une parcelle (refus au-delà du maximum).
- Laisser une plante passer par les 3 étapes et éclater : elle disparaît de la Hierarchy.
- Arroser une graine sur deux plantées côte à côte : l'arrosée mûrit la première.
- Survoler une graine (rien), puis un blé mûr : il est aspiré, le score monte. Changer le format de la vue Game : le score reste en haut à gauche.
- Montrer les sons, les particules et le vol stationnaire du drone ; ouvrir la pause (Échap) et les réglages du son.
- Remplir la soute, déposer dans la charrette (bonus), recharger l'eau et les graines.

- Finir une partie (objectifs baissés dans l'Inspector pour la démo) : écran de fin, nouveau record, puis Menu principal : le meilleur score s'affiche.
- Montrer la build Windows et la page itch.io.